Meridian Brokerage Group

Commercial Insurance Brokerage Platform — End-to-End System & Solution Architecture Combining UI, API, AI Subsystem, and Persistence

FRONTEND LAYER
React 19 + Vite 8
TypeScript 6, Tailwind CSS v4, TanStack Query v5

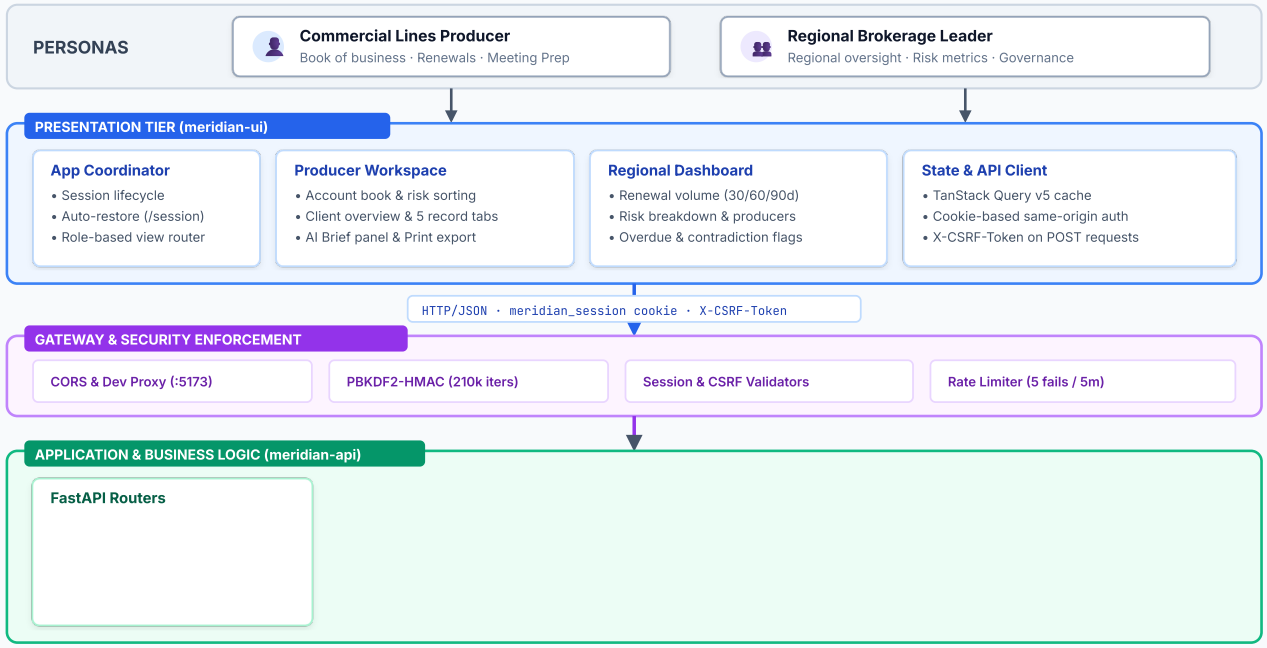
BACKEND CORE
FastAPI + Python 3.12
Uvicorn ASGI, Pydantic v2 Models, Session Auth

INTELLIGENCE & AI
Dual-Model Agent
GPT-4o + Gemini Flash + Offline Local Fallback

PERSISTENCE & DB
SQLite 3 / Postgres
ACID Relational Engine, Foreign Keys, Audit Trail

1. End-to-End Solution Architecture Diagram

The diagram below depicts the unified architecture bridging client presentation, security gateway, application service controllers, the AI inference pipeline, and relational storage.



`/auth` (login, session, logout) `/accounts` (books, records) `/portfolio` (regional metrics) `/admin/audit` (telemetry) Scoping & Rules Engine • Row scope: owner vs region • Column grants (policies, claims) • Rules R1-R10 (claims, dates) • Evidence confidence scoring • Risk roll-up (High/Med/Low) ClientIntelligenceAgent Subsystem Prompt & Verification • Structured Context Builder • Pydantic Output Validator • Claim Completeness Check Multi-Tier Resilience 1. Primary: gpt-4o gateway 2. Fallback: gemini-2.5-flash 3. Offline: Deterministic fallback PERSISTENCE TIER (SQLite / PostgreSQL) Security & Sessions users · auth_credentials auth_sessions · failures security_events · rules Brokerage Records accounts · contacts · policies claims · billing · cases documents · commitments Intelligence & Audits signals · evidence links contradictions · briefs audit_log (tokens, costs) EXTERNAL AI GATEWAY LiteLLM / TCS GenAI Lab Gateway • Primary: genailab-maas-gpt-4o • Secondary: gemini-2.5-flash • Endpoint: https://genailab.tcs.in/v1

2. Technology Stack Matrix Across All Layers

The following matrix details the active technologies, version baselines, and architectural roles across all six core tiers of the Meridian Broker solution:

ARCHITECTURAL LAYER	TECHNOLOGY & VERSION	CLASSIFICATION	ARCHITECTURAL ROLE & FUNCTIONALITY
Client Presentation `meridian-ui`	**React** `v19.2.8` **TypeScript** `~6.0.2` **Vite** `v8.3.0`	Frontend Core	Single-page application featuring fast HMR, strict type checking, and modular component architecture split into Producer and Regional Leader views.
Styling & Design System	**Tailwind CSS** `v4.3.3` `@tailwindcss/vite` **Lucide React** `v1.55.0`	UI Design	Compile-time CSS generation without runtime overhead; consistent enterprise-grade iconography for status badges, risks, and severity indicators.
Client State & Query	**TanStack React Query** `v5.104.1` **Zustand** `v5.0.15`	State Mgmt	Asynchronous server state caching, automatic query deduplication, background invalidation, and lightweight local store for user session management.
Visualizations	**Recharts** `v3.10.1`	Analytics UI	Declarative SVG charts displaying regional renewal volume, risk tier distribution, and historical loss trends.
Ingress & Gateway	**Vite API Proxy** (Local) **Reverse Proxy / Ingress** (Prod)	Ingress Tier	Proxies `/auth`, `/accounts`, `/portfolio`, and `/admin` to API port 8000; provides same-origin cookie protection and CORS isolation.
Backend API Core `meridian-api`	**FastAPI** `⩾0.143.0` **Uvicorn** `⩾0.54.0` **Python** `⩾3.12`	App Server	Asynchronous ASGI REST application with automatic OpenAPI specs, dependency injection, and sub-50ms query execution times.
Data Serialization	**Pydantic** `v2.x`	Data Models	Strict runtime validation and serialization for domain entities (Account, Policy, Claim, Billing, Brief) with Decimal arithmetic.
Identity & Security	**PBKDF2-HMAC-SHA256** **Opaque Session Cookies** **CSRF Token Tokens**	Security	210,000 hash iterations with 16-byte random salt. Revocable session tokens in SQLite. Mandatory `X-CSRF-Token` on mutating methods. Brute force rate limiting (5 attempts/5m).
Access Governance	**Scoping Guard (access.py)**	RBAC / ABAC	Row-level filtering (Producers: assigned book; Leaders: regional book) and fine-grained resource permission columns (policies, claims, billing, cases, documents).
Intelligence Rules	**Deterministic Engine (intelligence.py)**	Business Logic	Executes rules R1-R10 (open claims near renewal, PAS vs CRM date discrepancies, pending endorsements, overdue commitments, stale activity) with corroboration scoring.
AI Inference & Agent	**OpenAI Python SDK** `⩾1.50.0` **HTTPX** `⩾0.27.0` **LiteLLM Gateway**	AI Subsystem	Communicates with TCS GenAI Lab Gateway; dual-model strategy: Primary `genailab-maas-gpt-4o`, Fallback `gemini-2.5-flash`.
Agent Resilience	**Deterministic Local Generator**	Reliability	Zero-dependency offline engine that guarantees structured brief generation if external AI gateways are unreachable.
Persistence Engine	**SQLite 3** (Local dev) **PostgreSQL 15+** (Enterprise Cloud)	Database	Relational database with strict foreign keys, multi-table joins, atomic transactions, and audit logs. Complete Postgres schema and migration files included.

3. Key Architectural Workflows

🔐 Session & Security Flow

1. User posts credentials to `/auth/login`.
2. Backend checks rate limits in `login_failures`.
3. PBKDF2 salt and hash checked against DB.
4. High-entropy session token generated, hashed, and stored in `auth_sessions`.
5. Returns HTTP-only cookie + CSRF token.
6. Mutating calls validate `X-CSRF-Token`.

⚡ Deterministic Rules Engine

1. Validates user resource grants for the account.
2. Scans active policies, claims, billing, cases.
3. Applies R1-R10 rules against current business date.
4. Computes evidence confidence: `Score = min(0.98, Base + 0.10 * Corroboration)`
5. Date conflicts trigger human-review flag.
6. Aggregates account tier: High, Med, or Low.

🤖 AI Brief & Fallback Pipeline

1. Builds structured input payload (claims, policies, signals).
2. Calls Primary LLM (`gpt-4o`) via LiteLLM.
3. If timeout/failure, calls `gemini-2.5-flash`.
4. If gateway offline, uses local deterministic fallback.
5. *Completeness Guard:* Asserts open claims cited.
6. Records tokens, latency, cost in `audit_log`.

4. Data Architecture & Relational Entity Model

The persistence tier is divided into three logical domains: Identity & Security, Brokerage Business Domain, and Intelligence & Audit Observability.

DOMAIN AREA	TABLES	SCHEMA DESCRIPTION & RELATIONSHIPS

Security & Sessions	`users`
	`auth_credentials`
	`auth_sessions`
	`login_failures`
	`security_events`
		Maintains user identities, email addresses, assigned persona (`producer` vs `regional_leader`), and geographic regions. Hashed credentials store hex salts with PBKDF2 iterations. `auth_sessions` tracks token hashes, CSRF tokens, and expiry epochs. `login_failures` tracks attempts for brute-force lockout. `security_events` logs forensic actions.

Access Governance	`access_rules`	Defines account-level access grants (`allowed=1`) and fine-grained resource visibility flags: `can_view_policies`, `can_view_claims`, `can_view_service_cases`, `can_view_billing`, `can_view_documents`.

Commercial Brokerage Domain	`accounts`
	`contacts`
	`policies`
	`claims`
	`billing`
	`service_cases`
	`documents`
	`endorsements`
	`activities`
	`commitments`
		Core entity tables storing commercial client portfolios. `accounts` links to `owner_user_id` and records annual premium, risk tier, and renewal timeline. `policies` tracks carriers, coverage lines, and dual renewal dates (PAS vs CRM). `claims` tracks open/closed status, reserve amounts, loss dates, and severity. `commitments` stores producer promises and deadlines.

Intelligence & Audit Subsystem	`signals`
	`evidence`
	`contradictions`
	`briefs`
	`audit_log`
		Maintains curated and rule-computed risk/opportunity signals. `evidence` provides foreign-key traceability to source records. `contradictions` logs data discrepancies for human review. `briefs` preserves versioned structured JSON briefs. `audit_log` tracks LLM token usage, cost in USD, latency in ms, and completeness flags.

5. Non-Functional & Operational Architecture

QUALITY ATTRIBUTE	IMPLEMENTATION MECHANISM	OPERATIONAL BENEFIT
High Availability & Resilience	3-tier cascade: Primary LLM $\to$ Fallback LLM $\to$ Deterministic local brief generator.	Guarantees zero-downtime operation for field producers heading into renewal discussions even during gateway outages.
Zero-Trust Authorization	Dual-scope SQL queries enforcing `owner_user_id` (Producers) or `region` (Leaders) + `access_rules`.	Prevents unauthorized cross-book or cross-region data leakage. Enforced centrally at API layer.
Sub-50ms Query Performance	Indexed relational queries in SQLite/PostgreSQL combined with TanStack Query client caching.	Instant dashboard rendering, search, filtering, and tab switching with zero redundant API calls.
AI Observability & Cost Control	Granular per-generation telemetry logged to `audit_log` (tokens in/out, calculated USD cost, latency).	Complete visibility into enterprise AI consumption, budget allocation, model performance, and audit compliance.
Enterprise Cloud Portability	Standardized FastAPI ASGI app, clean Docker separation, and full PostgreSQL schema script provided.	Frictionless migration from local developer environments to AWS (ECS/EKS, RDS PostgreSQL, CloudFront, ALB).